1. Gyakorlat

Algoritmusok és adatszerkezetek 2.

Intro

Alapok

- Bemutatkozás
- Félév rendje
- Követelmények
  - ZH-k
  - Kvízek: 2 hét van rá
  - Jelenlét: be kell járni
  - Szorgalmik: beadás digitálisan, xls-ben készítve

Miről lesz szó?

- Adattömörítés
- AVL fák, B fák
- Gráfok és ábrázolásuk
- Gráfalgoritmusok: bejárás, útkeresés, feszítőfa, topologikus rendezés
- Mintaillesztések

Adattömörítés

Egy kis elmélet

- Kódoláselmélet adatok különböző reprezentációjával és azok közötti átalakításokkal foglalkozik
- Forráskódolás az adott alak hosszát vizsgálja: adott mennyiségű információt mekkora mennyiségű adattal lehet tárolni
- Cél a rövidebb reprezentáció, tehát beszélhetünk információ- vagy adattömörítésről.
- Információ alapegysége: r = 2 és így a kódszavaink a T = {’0’, ’1’} ábécé feletti szavak lesznek

Betűnkénti kódolás

- eredeti ∑ ábécé feletti adatot betűnként egy $\sum \rightarrow C \subset T^*$ kölcsönösen egyértelmű (bijektív) leképezéssel készítjük el
  - Pl: ascii
- Kódolandó állományokat karaktersorozatnak tekintjük
- A tömörítés célja az adatok kisebb helyen történő tárolásának biztosítása.

Naiv módszer

- A tömörítendő szöveget karakterenként, fix hosszúságú bitsorozatokkal kódoljuk.
- A kódot egyenletes kódnak nevezzük, ha a kód szavainak hossza egyenlő.
- A naiv módszer egyenletes kódot használó betűnkénti kódolás.
- $\sum = \langle \sigma 1, \sigma 2, \ldots, \sigma d \rangle$ az ábécé.
- A $\sum$ ábécé feletti kódolt adat akkor lesz a legkisebb, ha a kódszavak közös hossza a legkisebb

1. Feladat: Milyen hosszú legyen a kódszó?

- $|T| = r$ és $|\sum| = d$
- Az egyes karakterek legkevesebb $\lceil \log_r d \rceil$ hosszal kódolhatóak naiv módszer segítségével.
- $\lceil \log_2 d \rceil$ bittel kódolható, ugyanis $\lceil \log2\ d \rceil$ biten
- $2^{\lceil \log2 d \rceil}$ különböző bináris kód ábrázolható
- $2^{\lceil \log2 d \rceil} \geq d > 2^{\lceil \log2 d \rceil - 1}$, azaz $\lceil \log2\ d \rceil$ biten ábrázolható d-féle különböző kód, de eggyel kevesebb biten már nem.

2. Feladat

Kódoljuk az ABRAKADABRA szöveget fix hosszúságú kóddal

Milyen hosszú lesz a kódolt szöveg?

Mit nyerünk ezzel, feltéve, ha eredetileg az egyes karaktereket 8 biten ábrázoltuk eredetileg?

2. Feladat - megoldás

- d = 5 - ABC betűinek száma
- n = 11 - szöveg hossza
- tömörített kód hossza: 11 * ⌈ log 5 ⌉ = 11 * 3 = 33 bit
- Mit nyerünk:
  - Eredeti szöveg: 11*8 = 88
  - 33/88 = 3/8 = 37,5% avagy 62,5% megtakarítás
- Kódok: A 3 bites kódok közül tetszőleges 5 kioszthatóaz 5 betűnek.
- Nem számoltuk bele: a kódtábla átküldését

2. Feladat - folytatás

Az ABRAKADABRA szöveg kódtáblázata lehet a következő:

karakter	kód
A	000
B	001
D	010
K	011
R	100

Ezzel a kódtáblázattal a tömörített kód a következő lesz:

000001100000011000010000001100000.

Huffmann-kód

Huffmann kód

- Hatékony módszer az adatállományok tömörítésére
- Ötlet: a gyakrabban előforduló karakterek kódja legyen rövidebb
- Mohó algoritmus:
  - Az adott pillanatban legjobbnak tűnő lépést választja
  - Itt optimális (de nem minden problémából jó a mohó stratégia)
- Egy táblázatot használ az egyes karakterek előfordulási gyakoriságára

3. feladat

Mi a probléma az alábbi stratégiával?

Számoljuk meg egy input szövegben a karakter előfordulások számát. A karakterek gyakoriságuk szerint csökkenő sorrendben kapják a 0, 1, 00, 01, 10, 11, 000, 001, … sorozatban szereplő kódokat. A végén a kódolt szöveggel együtt a kódtáblát is átküldjük.

3. Feladat - megoldás

A kódtábla és a kód alapján nem tudunk dekódolni.

Pl:

Kód: 0001

Eldönthetetlen, hogy:

AAAB vagy ACB vagy CD vagy CAB

Betű	Db.	Kód
A	4	0
B	3	1
C	2	00
D	1	01

4. Feladat - Prefix kód

- Próbáljuk dekódolni az alábbi kódot a megadott kódtábla alapján

  01100011100111

  Képesek vagyunk erre? Próbáljuk indokolni válaszunkat.

Betű	Kód
A	1
B	00
C	01

4. Feladat - Megoldás

C A**B** C **A**B **AA**A
011**00**0**1**1**00**1**11**

Miért volt ez lehetséges?

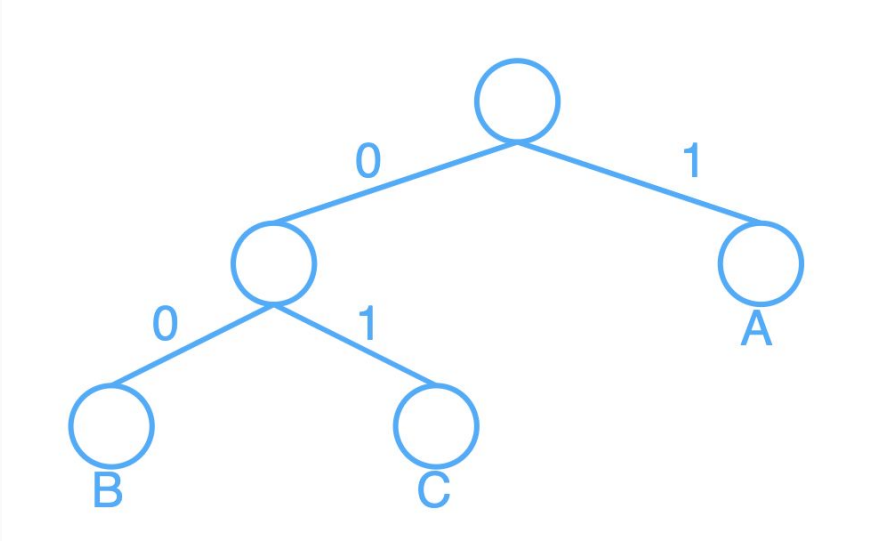
Prefix (mentes) kód, nem kezdődik semelyik kód egy másik kóddal.

Pl: 0, 01 kódok nem jók, mert egy 0-t olvasva nem tudjuk, hogy tovább kell-e olvasnunk.

Betű	Kód
A	1
B	00
C	01

Kódtábla reprezentációja bináris fával

Betű	Kód
A	1
B	00
C	01



Huffmann kód algoritmusa

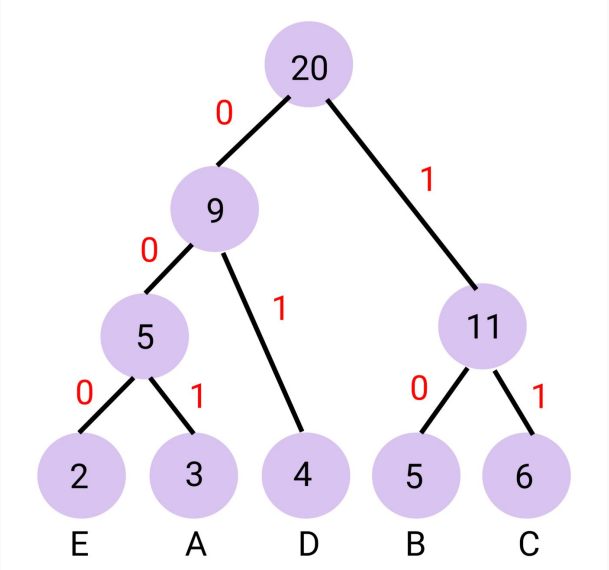
1. Olvassuk végig a szöveget és határozzuk meg az egyes karakterekhez tartozó gyakoriságokat.
2. Hozzunk létre minden karakterhez egy csúcsot és helyezzük el azokat egy (min) prioritásos sorban a gyakoriság, mint kulcs segítségével.
3. Egymás után hajtsuk végre:
    a. Vegyünk ki két csúcsot a prioritásos sorból és hozzunk létre számukra egy szülő csúcsot.
    b. A szülő-gyerek éleket címkézzük nullával és eggyel a gyakoriságnak megfelelő sorrendben.
    c. Helyezzük el a szülő csúcsot a prioritásos sorba gyerekei gyakoriságának összegét használva kulcsként.
4. Ismételjük meg az előző pontot, ha több mint egy csúcs szerepel a sorban.
5. Olvassuk ki a karakterekhez tartozó kódszavakat a kódfából.
6. Olvassuk végig újra a bemenetet és kódoljuk azt karakterenként.

Huffmann kód / 2.

- A tömörítendő fájlt, illetve szöveget kétszer olvassa végig.
- A kódfa szigorúan bináris fa.
- A tömörített fájl a kódfát is tartalmazza.

Kód:

E:000
A:001
D:01
B:10
C:11



Huffman kód kitömörítés

1. Mindegyik karakter kinyeréséhez a kódfa gyökerétől indulunk.
2. Majd a tömörített kód sorban olvasott bitjei szerint 0 esetén balra, 1 esetén jobbra lépünk lefelé a fában, mígnem levélcsúcshoz érünk.
3. Ekkor kiírjuk a levelet címkéző karaktert, majd a Huffman-kódban a következő bittől és újra a kódfa gyökerétől folytatjuk, amíg a tömörített kódon végig nem érünk.

Gyakorlófeladat

Írjunk algoritmust, amely kiírja egy bináris fa összes levélcsúcsában lévő kulcsértéket valamint a gyökértől oda vezető utat úgy, hogy a balgyereket mindig a 0, a jobbgyereket mindig az 1-es címke jelzi.

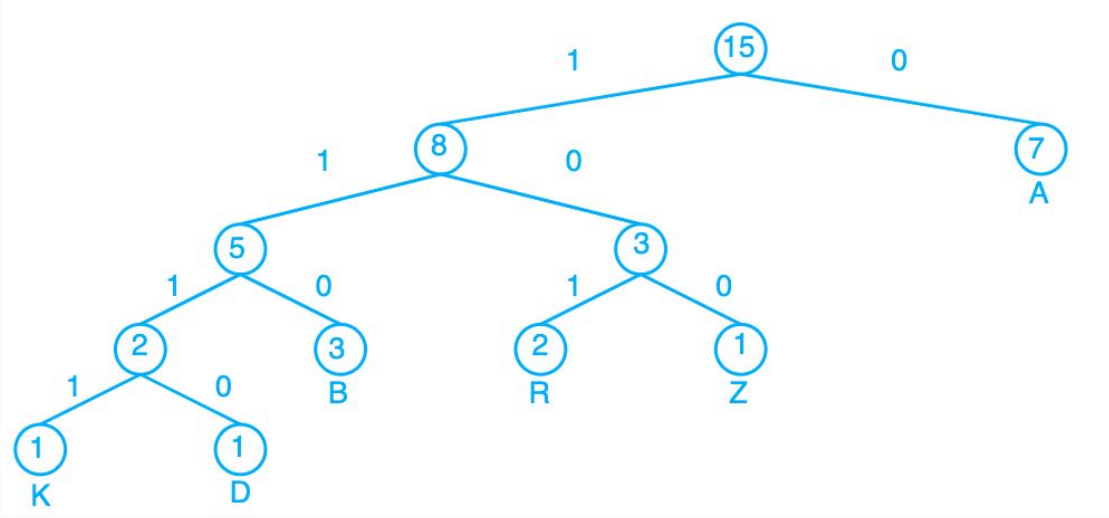
Példa

Kódoljuk a következő szöveget Huffmann kódolással:

AZABBRAKADABRAA

Megoldás

karakter	gyakoriság	kód
A	7	0
B	3	110
D	1	1110
K	1	1111
R	2	101
Z	1	100



5. Feladat - házi feladat

Írjuk meg az előző félévben tanult prioritásos sor, bináris fa és egyszerű asszociatív szótár adatszerkezet segítségével struktogramként a Huffmann kódolás algoritmusát, amely inputként egy szöveget kap és az outputra kiírja az egyes karakterek kódját ***Karakter:bináris kód;*** formában.

6. Feladat

Keressünk olyan szöveget, ahol a Huffmann kódolás pontosan annyi bittel történik, mint a naiv módszerrel.

Megoldás

Például: AB
Általában: ha 2 hatvány a betűk száma és minden betűből pontosan ugyanannyi van, akkor ugyanannyi biten lesz kódolva minden betű.

7. Feladat

Hogyan általánosítanánk a Huffmann algoritmust ternális kódszavakra? Tehát ha a kódolásban a {0,1,2} jeleket használhatjuk.

7. Feladat - megoldás

Egyszerűen gondoljuk át, hogy min kell és min nem kell változtatni:

1. Olvassuk végig a szöveget és határozzuk meg az egyes karakterekhez tartozó gyakoriságokat.
2. Hozzunk létre minden karakterhez egy csúcsot és helyezzük el azokat egy (min) prioritásos sorban a gyakoriság, mint kulcs segítségével.
3. Egymás után hajtsuk végre:
   a. Vegyünk ki ~~két~~ **három** csúcsot a prioritásos sorból és hozzunk létre számukra egy szülő csúcsot.
   b. A szülő-gyerek éleket címkézzük ~~nullával és eggyel~~ 0, 1 és 2-vel gyakoriságnak megfelelő sorrendben.
   c. Helyezzük el a szülő csúcsot a prioritásos sorba gyerekei gyakoriságának összegét használva kulcsként.
4. Ismételjük meg az előző pontot, ha több mint egy csúcs szerepel a sorban.
5. Olvassuk ki a karakterekhez tartozó kódszavakat a kódfából.
6. Olvassuk végig újra a bemenetet és kódoljuk azt karakterenként.

Lempel-Ziv-Welch/LZW módszer

Alapok

- Az LZW kódolás egy kezdeti kódtáblát bővít
- A szöveg bizonyos szavaiból szótárat épít
  - Egyre hosszabb már „látott” szavakhoz rendel új kódszót
- Ez szavak egy halmaza, amit S-sel jelölünk
- A szótárról három dolgot tételezünk fel:
  - az egybetűs szavak mind szerepelnek benne;
  - ha egy szó benne van a szótárban, akkor annak minden kezdődarabja is benne van;
  - a tárolt szavaknak fix hosszúságú kódjuk van; $x \in S$ szó kódját c(x) jelöli.

LZW Algoritmus

1. Olvassuk a bemenő állomány következő betűjét K-ba.
2. Ha az előző olvasási kísérlet sikertelen volt (vége a bemenetnek), akkor írjuk ki c(z)-t, és álljunk meg.
3. Ha a zK szó is S-ben van, akkor z ← zK, és menjünk vissza (1)-re.
4. Különben (ha zK $\notin$ S) írjuk ki c(z)-t, tegyük a zK szót S-be. Legyen z ← K, majd menjünk vissza (1)-re.

Példa

Legyen a tömörítendő szöveg: ABABABAACAACCBBAAAAAAAAA

Kezdeti kódtábla

Karakter	kód
A	1
B	2
C	3

Megoldás

- A kódolás során a bemenetet pontosan egyszer olvassuk végig úgy, hogy mindig a már ismert (kóddal rendelkező) leghosszabb szót keressük. Ha megtaláltuk, akkor
  - kiírjuk a talált szó kódját a kimenetre, és
  - bővítjük a kódszavak halmazát az αK szó képével, ahol az α a talált szó és K a következő karakter.
- Kezdetben a leghosszabb „ismert” szó az A, ennek kódja 1 és az új kóddal rendelkező szó az AB a 4 kóddal.
- A kódolás folyamán ezt az lépést ismételjük, amíg a szöveg végére nem érünk.

kód	aktuális szó	következő karakter	új kód
1	A	B	4
2	B	A	5
4	AB	A	6
6	ABA	A	7
1	A	C	8
3	C	A	9
1	A	A	10

Folyt.

8	AC	C	11
3	C	B	12
2	B	B	13
5	BA	A	14
10	AA	A	15
15	AAA	A	16
15	AAA	-	-